

1.7 OPERATING-SYSTEM STRUCTURES

Internally, different operating systems vary greatly from one another in their structure. Therefore, before designing an operating system, the designers must have clarity about which type of system is desired. For this, the designers are required to study the operating systems from several different viewpoints, for example, how it is disassembled into its various components and how those components are interconnected, which services it offers and how they are provided, what kind of interface it provides to the users and programmers, etc. These aspects of operating systems are below.

1.7.1 System Components

An operating system is very a large and complex system.

To make its designing easier, it is partitioned into smaller parts where each part refers to a well-defined portion of the system and has defined inputs, outputs, and functions.

Though the structure of all systems may differ, the common goal of most systems is to support the system components.

Process Management

The basic concept supported by almost all the operating systems is the process.

A process is a program under execution or we can say an executing set of machine instructions.

A program by itself does nothing; it is a passive entity. In contrast, a process is an active entity that executes the instructions specified in the program.

It is associated with a program counter that specifies the instruction to be executed next.

The instructions of a process are executed sequentially, that is, one after another until the process terminates.

A program doesn't need to have only a single process; rather it **may be associated with many processes**.

However, the **different processes associated with the same program are not treated as separate execution sequences**.

Furthermore, **a process may spawn several other processes (known as child processes)** during execution. These child **processes may in turn create other child processes**, resulting in a **process tree**.

A process is intended to **perform a specific task**. To do its intended task, each process **uses some resources during its lifetime**, such as **memory, files, CPU time, and I/O devices**.

These resources can be **allocated to the process either at the time of its creation or during its execution**.

In addition to resources, **some processes may also need certain input when they are created**.

For example, if a **process is created to open a specific file**, then it is required to provide the desired **file name as input** to the process so that the process could execute the appropriate instructions and system calls to accomplish its task.

After the **process is terminated**, the reusable **resources** (if any) are **reclaimed** by the operating systems.

A process can be either **a system process executing the system's code** or **a user process executing the user's code**. Usually, a system contains **a collection of system and user processes**. These processes can be made to **execute concurrently by switching the CPU among them**.

In relation to process management, the responsibilities of an operating system are as follows:

- **to create and delete processes** (including both user and system processes),
- **to suspend the execution of a process temporarily and later resume it**,
- **to facilitate communication among processes** by providing communication mechanisms, and

- to provide mechanisms for dealing with deadlock.

Memory Management

A computer system usually uses main memory and secondary storage.

The main memory is central to the operation of a computer system.

It is a huge collection of words or bytes, which may range from hundreds of thousands to billions in size, and each byte or word has a unique address.

It holds the instructions and data currently being processed by the CPU, the result of intermediate calculations, and the recently processed data.

It is the only storage that is directly accessible to the CPU.

Whenever a program is to be executed, it is allocated space into the main memory. As it executes, it accesses the data and instructions from the main memory. After the program has been executed, the memory space allocated to it is de-allocated and declared available for some other process. This is the case of single process in memory which leads to inefficient memory utilization. To improve the utilization, multiprogramming is used in which multiple programs are allowed to reside in the main memory at the same time.

However, in this case, the operating system needs more sophisticated memory management techniques as compared to the single-user environment. It is the responsibility of the memory manager to manage memory between multiple programs in an efficient way. In relation to memory management, the responsibilities of an operating system are as follows:

- to allocate and deallocate memory space as and when required,
- to make a decision on which of the processes (ready for execution) should be allocated memory when it is available, and
- to keep track of the parts of memory that have been allocated and to

which processes.

Usually, the **capacity of the main memory is limited** and not enough to accommodate all data and programs in a typical computer. Moreover, **all the data stored is lost when power is lost** or switched off. Thus, it is **required to use secondary storage** in a computer system to back up main memory.

The **most commonly used** secondary storage in computer systems is the **disk**. It **stores the most programs**, such as compilers, sort routines, assemblers, etc. These programs are **loaded into the main memory when needed and otherwise kept stored on the disk**. Thus, it is important for a computer system that the disk storage must be used efficiently. In relation to disk management, the responsibilities of an operating system are as follows:

- to allocate space on disk,
- to manage the unused (free) space available on disk, and
- to perform disk scheduling.

File Management

Another important component of all the operating systems is file management, which **deals with the management and organization of various files in the system**. As we know that a computer system consists of various storage devices such as hard disks, floppy disks, compact discs, and so on, the operating system **provides an abstract view of these devices by hiding their internal structure** so that the users can directly access the data (on physical devices) without exactly knowing where and how the data is actually stored.

The operating system defines **a logical storage unit** known as a **file**, and all the **data is stored** in the form of files. Each file is associated with some **attributes** such as its **name, size, type, location, date and time, etc.** The users can perform various **operations** on files such as **create, delete, read, write, open, seek, rename, append, and close**. Operating system handles these operations **with the help of system calls**.

To organize the files in a systematic manner, the operating system

provides the concept of directories. A **directory** can be defined as a way of grouping files together. The directories are organized in a hierarchical manner, which allows users to have subdirectories under their directories, thus making the file system more logical and organized. In relation to file management, the responsibilities of an operating system are as follows:

- to create and delete files and directories,
- to back up the files onto some stable storage media,
- to map files onto secondary storage, and
- to provide primitives that enable one to manipulate the contents of files and directories.

I/O Management

A computer system consists of several I/O devices such as keyboard, monitor, printer, and so on. It is the responsibility of an operating system to control and manage these devices. The operating system also provides a device-independent interface between the devices and the users so that the users can issue commands to use these devices without actually knowing how their commands are being executed. To hide the details of different devices, operating system designers let the kernel to use device-driver modules, which present a uniform device-access interface.

Protection and Security

In multiprogramming or multiuser systems where multiple processes from many users are allowed to execute concurrently, some protection mechanisms are needed to prevent the processes from interfering into one another's activities.

These mechanisms must ensure that the resources like memory segments, files, CPU, etc., can be accessed by only those processes which have been granted permission by the operating system. For example, to ensure memory protection, the operating system provides memory addressing hardware that ensures that each process executes in its own address space. In general, **protection** refers to any mechanism used to control the access of processes, programs, or users

to the resources defined by the computer system. An **unprotected** resource is liable to be **misused by any unauthorized user**.

Despite the fact that protection is ensured, a system may tend to fail and allow unauthorized access. To understand how it can happen, consider a user whose authentication information has been stolen. Now, any unauthorized person can use the stolen information to copy or modify that user's data even though the memory and file protection are working. This occurs due to the lack of **security that ensures the protection of system against attacks from insiders as well as external attacks**. Some of the common security attacks include **viruses, worms, trap doors, identity theft, denial of service (DoS) attacks**, etc. **Prevention from these attacks** is considered as the responsibility of operating systems on some systems, while on other systems, prevention **is implemented using policies or additional software**.

1.7.3 Operating-system Services

Almost **all the user programs need an environment** in which they can be executed. In addition, **they need a set of services** using which the burden of programming reduces and it becomes easier. For instance, programmers should not be bothered about how memory is allocated to their programs, where their programs are loaded in memory during execution, how multiple programs are managed and executed, how their programs are organized in files to reside on disk, etc. **Providing this environment** in which programs can be executed **and the set of services** to user programs **are the operating system responsibilities**. One set of operating-system services provides functions to help the user. These services include the following:

- **User interface:** Providing a **user interface (UI)** to interact with users is essential for an operating system. This interface can be in one of the several forms. One is the **command-line interface**, in which **users interact with the operating system by typing commands**. Another is the **batch interface**, in which **several commands and directives to control those commands are collected into files that are then executed**. Another is the **graphical user interface (GUI)**, in which **users interact with the system with a pointing device**, such as a mouse.
- **Program execution:** The system must **allocate memory** to the

user programs, and then **load these programs** into memory so that they can be executed. The **programs** must be able to **terminate either normally or abnormally**.

- **I/O operations:** Almost all the programs require I/O involving a file or an I/O device. For efficiency and protection, **the operating system must provide a means to perform I/O instead of leaving it for users to handle I/O devices directly**.
- **File-system manipulation:** Often, **programs need to manipulate** files and directories, such as creating a new file, writing contents to a file, deleting or searching a file by providing its name, etc. Some programs may **also need to manage permissions** for files or directories to allow or deny other programs requests to access these files or directories.
- **Communication:** **A process executing in one computer may need to exchange information with the processes executing on the same computer or on a different computer connected via a computer network**. The information is moved between processes by the operating system.
- **Error detection:** There is **always a possibility of occurrence of error** in the computer system. Error may occur in the **CPU, memory, I/O devices, or in user program**. Examples of errors include an attempt to access an illegal memory location, power failure, link failure on a network, too long use of CPU by a user program, etc. **The operating system must be constantly aware of possible errors, and should take appropriate action in the event of occurrence of error to ensure correct and consistent computing**.

As we know, multiple programs may be executed concurrently each of which may require multiple resources during their execution. Therefore, providing another set of services that help allocating resources to programs in some order is necessary for an operating system. These services exist not for helping user, instead to ensure the efficient and secure execution of programs.

- **Resource allocation:** In case of multiprogramming, many programs execute concurrently, each of which require many different types of resources, such as CPU cycles, memory, I/O devices, etc. Therefore, in such an environment, **operating system must allocate resources to programs in a manner such that resources**

are utilized efficiently, and no program should wait forever for other programs to complete their execution.

- **Protection and security:** Protection involves ensuring controlled access to the system resources. In a **multiuser** or a networked computer system, the owner of information may want to protect information. When several processes execute concurrently, a process should not be allowed to interfere with other processes or with the operating system itself. Security involves protecting the system from unauthorized users. To provide security, each user should authenticate himself or herself before accessing system resources. A common means of authenticating users is username/password mechanism.
- **Accounting:** We may want to keep track of the usage of system resources by each individual user. This information may be used for accounting so that users can be billed or for accumulating usage statistics, which is valuable for researchers.

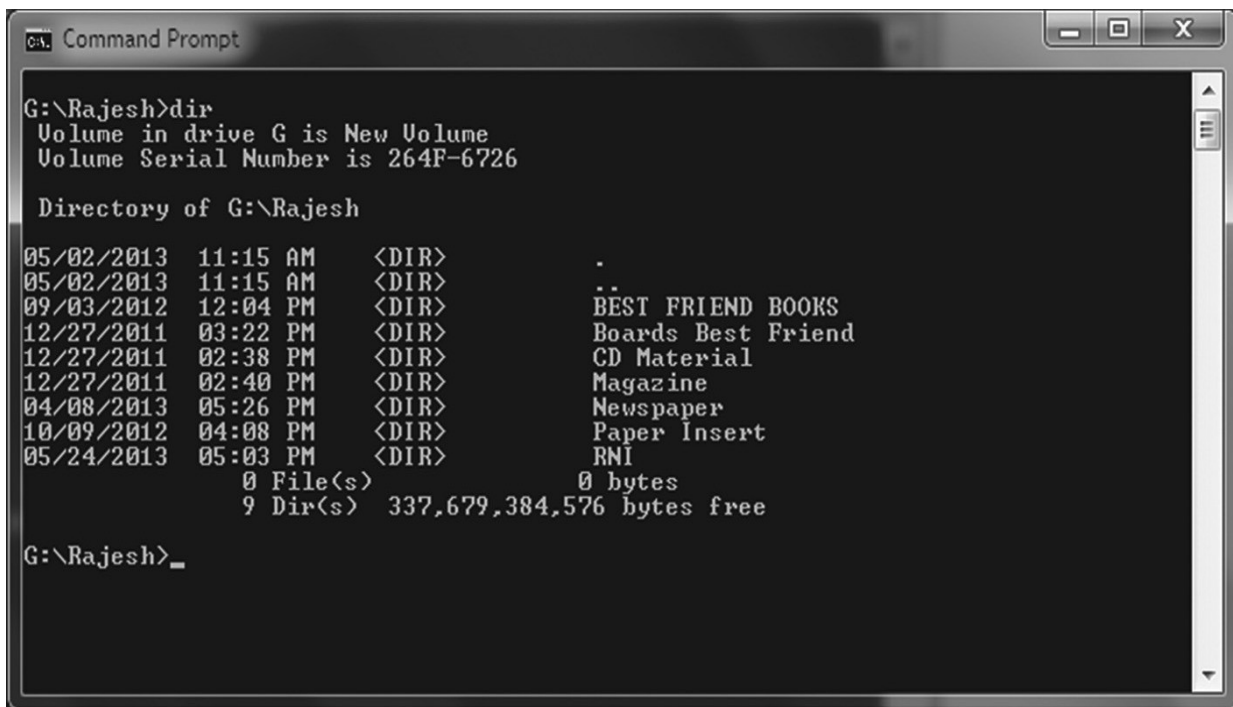
1.7.4 User Operating-System Interface

Providing an interface to interact with the users is essential for an operating system. Earlier operating systems provide users with the command-line interface or character-based interface. This interface enables users to interact with the operating system by entering commands to which it responds. On the other hand, most operating systems nowadays provide graphical user interface (GUI) in addition to character-based interface. GUI enables users to interact with the operating system by clicking mouse buttons.

Command-line Interface

As mentioned, this interface enables users to interact with the operating system by typing commands. These commands are then interpreted and executed in order to provide response to the user. The MS-DOS is the most commonly used operating system that provides command-line interface. Figure 1.10 shows MS-DOS command-line interface. Some operating systems provide more than one command-line interface; therefore, on such systems, command-line interfaces are called **shells**.

For example, UNIX provides C shell, Bourne shell, Korn shell, etc.



```
C:\ Command Prompt

G:\Rajesh>dir
Volume in drive G is New Volume
Volume Serial Number is 264F-6726

Directory of G:\Rajesh

05/02/2013  11:15 AM    <DIR>          .
05/02/2013  11:15 AM    <DIR>          ..
09/03/2012  12:04 PM    <DIR>          BEST FRIEND BOOKS
12/27/2011  03:22 PM    <DIR>          Boards Best Friend
12/27/2011  02:38 PM    <DIR>          CD Material
12/27/2011  02:40 PM    <DIR>          Magazine
04/08/2013  05:26 PM    <DIR>          Newspaper
10/09/2012  04:08 PM    <DIR>          Paper Insert
05/24/2013  05:03 PM    <DIR>          RNI
               0 File(s)                0 bytes
               9 Dir(s) 337,679,384,576 bytes free

G:\Rajesh>_
```

so on. These commands can be implemented in two ways. In the first method, the code to execute the commands could be included in the interface itself. Now, whenever a command is issued by the user, the interface jumps to a section of its code and makes the appropriate system call. Since the code for the commands is included in the interface itself, adding new commands require changing the interface. In addition, the size of interface increases with each new command.

Alternatively, system programs can be developed that include the code for most commands. In this case, the interface has no idea about the command implementation; instead it just uses the command to locate the file and then loads and executes it. The UNIX operating system follows this approach; therefore, a command in UNIX would search a file with that name, load it in memory, and then execute it. For example, the command

```
cp file1.txt file2.txt
```

interprets `cp` as the file name, searches it, and loads it in the memory for execution. The command `cp` creates a copy of a given file. In the aforementioned command, `file1.txt` and `file2.txt` are the name of the source and the destination files, and they are treated as parameters during the execution of the command `cp`. It means a copy of `file1.txt` is created and named `file2.txt`.

With this approach, the new commands can be added to the system easily by creating new system files.

The names of the files will then serve the command name.

Since adding new files does not require changing the interface, the size of the interface remains unchanged and small.

Graphical User Interface

Since in command-line interface, users interact by issuing certain commands, there is always a chance of committing a mistake. For example, opening a file might require the user to enter the path where the file actually resides. Any mistake in entering the path will prevent the user to open the file.

An alternative and more user-friendly method to interface with the operating system is the graphical user interface (GUI). GUI provides a rectangular area of screen called **Window** in which files, programs, directories, and system functions are represented as small images or symbols called **icons**. In addition, various **Menus** are provided that list actions or commands to be performed by the user.

Such interface enables users to interact with the operating system by moving the mouse to position the mouse cursor and clicking on some icon or menu option. Depending upon the position of the mouse cursor and the button (left or right) clicked on the mouse, some action is performed such as opening of a file, execution of a program, appearing of a menu, etc.

UNIX, Apple Macintosh (Mac OS), and various versions of Microsoft Windows including version 1.0 are some examples of operating systems that provide GUI. Many operating systems provide users with both command-line interface and GUI, and it depends on a person's personal choice to choose from and work with. Many UNIX programmers prefer to use command-line interface because it is faster to work

with and provides powerful capabilities, whereas, almost all Window users use GUI. GUI of Microsoft Windows 7 is shown in [Figure 1.11](#).



Fig. 1.11 GUI of Microsoft Windows 7

1.7.5 System Calls

A **system call** is a controlled interface that allows a user program to request services from the operating system kernel. Because user programs lack direct access to hardware and critical resources for security reasons, system calls act as a secure gateway to perform privileged operations.

In other words, user programs take these services from operating system by making system calls. These calls are similar to a procedure call except that it switches the mode of execution from the user mode to the kernel mode and invokes the operating system. The operating system then determines what the user program actually asks or wants, performs the system call, and returns the control back to the instruction following the system call. The user program now again proceeds in the user mode.

To understand how system calls are used in the command-line interface, let us take an **example** of a program that opens a file and writes some data in it. Suppose the file name and the data to be written on it is provided by the user during the execution of the program. This

program performs a sequence of system call during its execution. First, it makes a system call to prompt a message on screen to ask the user to provide the file name and the data to be written on the file. Then, the user provides the file name and the data by typing it through keyboard, and reading this from keyboard again needs a system call. With having the file name, the program attempts to open the required file, for which another system call needs to be made. Once the file is opened, the data is written to the file (requires a system call) and the file is closed (another system call). Finally, a system call is performed to prompt the user with a message to inform that the task is completed successfully.

The earlier discussion explains the use of system calls during normal operation; however, error could occur during any operation. For instance, when the program attempts to open the file, an error such as file not found, hardware failure, file protection violation, etc., could occur. In this situation, the program cannot proceed with its normal behaviour, instead, it should prompt an appropriate message on the screen (a system call needed) and then terminate abnormally (another system call).

As now, it is clear that even the simple programs make a heavy use of operating system services through system calls. In general, **the services offered by these calls include creation and termination (or deletion) of processes, creation, deletion, reading, writing, opening, and closing files, management of directories, and carrying out input and output.** In fact, the set of services that are offered through system calls determine a significant part of the operating systems responsibilities. Here note that each system call may have same or different name in different operating systems.

Types of System Calls

All the system calls provided by an operating system can be roughly grouped into following five major categories:

- **Process management:** The system calls under this category include the **calls to create a new process, terminate a process, setting and retrieving process attributes (such as process priority, its maximum allowable execution time, etc.), forcing a process to wait for some time or some event to occur, etc.** Some of the commands that come under this category are create process, terminate process,

load, execute, end, and abort.

- **File management:** The system calls under this category include the calls to [create, delete, open, close, read, and write a file](#). Some of the commands that come under this category are create, delete, open, close, read, and write.
- **Device management:** The system calls under this category include [the calls to request for device, releasing it, and performing some operations \(such as read or write\) with the device](#). Some of the commands that come under this category are request, release, read, and write.
- **Information maintenance:** The system calls under this category include the [calls to return information about the system, such as system's current data and time, number of current users, version of operating system, amount of free memory, etc.](#)

Some of the commands that come under this category are time, date, get process attributes, and set process attributes.

- **Communications:** The system calls under this category include the [calls to open and close communication connection, reading and writing messages, etc.](#) The commands that come under this category are open and close.

1.7.5 System Programs

In addition to system calls, modern systems also provide a variety of system programs. [These programs act as an interface between the operating system and the application programs.](#)

[They provide an environment in which application programs can be developed and executed in a convenient manner.](#)

They can be classified into the following categories:

- **File management:** The system programs under this category provide commands such as [cut/copy, dump, list, print, etc.](#), to

perform various operations on files and directories.

- **File modification:** The system programs under this category **allow creating or modifying the contents of a file stored on disk or some other storage device**. **Text editor** is a system program that belongs to this category.
- **Communications:** The system programs under this category **enable communication among different users, processes, or systems by establishing virtual connections between them**. With the help of these programs, a user can **send messages to other users, log on some remote systems, or transfer data from other system to its own system**.
- **Status information:** The system programs under this category are **used to present the status of the computer system** such as **system date and time, number of users connected, CPU utilization, disk and memory usage, configuration information, etc.**
- **Programming language support:** Nowadays, several programming languages support different system programs such as **compilers, assemblers, and interpreters**. These system programs are generally **provided to the users along with the operating system**.
- **Program loading and execution:** **After a user program has been complied, it needs to be loaded in the main memory for execution**. The task of **loading a program into the memory** is performed by the **loader**, a system program. A system may provide different loaders including **absolute loaders, relocatable loaders, overlay loaders**, etc. In addition, the successful execution of the program also requires debugging, which is performed by **debugger**—another system program under this category.

1.7.6 System Structure

Every operating system has its own internal structure in terms of file arrangement, memory management, storage management, etc., and **the entire performance of the system depends on its structure**. The internal structure of operating system **provides an idea of how the components of the operating system are interconnected and blended into kernel**. The

various system structures that have evolved with time are as below:.

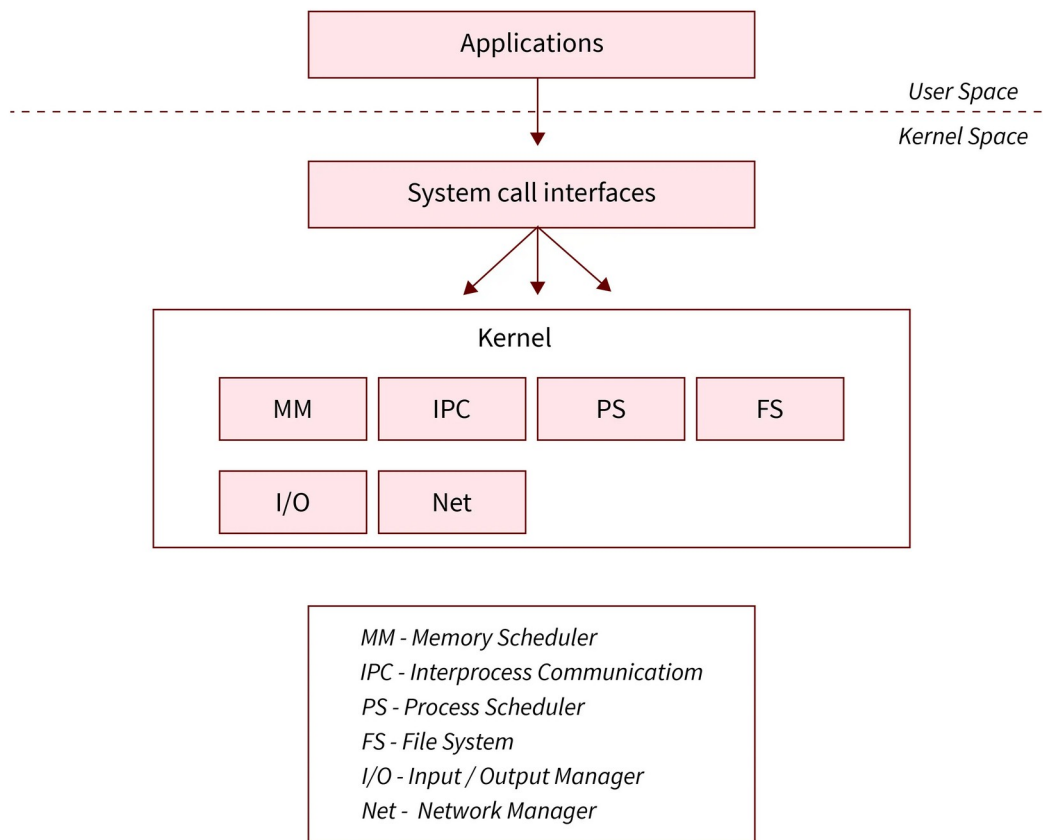
Simple Structure- Monolithic Architecture

Early operating systems were developed with an elementary approach without much concern about the structure.

In this approach, the structure of the operating systems was not well-defined.

The operating systems were **monolithic**, written as a collection of procedures where each procedure is free to call any other procedure.

- A monolithic OS is an operating system architecture where the entire operating system is working in kernel space and is alone in supervisor mode.
- Every component of the operating system is contained in the kernel and can directly communicate with any other (i.e., simply by using function calls).
- The kernel typically executes with unrestricted access to the computer system. Ex:- MS-DOS, OS/360,VMS and LINUX.



An example of operating systems designed with this approach is MS-DOS. Initially, **MS-DOS** was designed as a small-size and simple system, and with limited scope, but grew beyond its scope with time. It was designed with the idea of providing more functionality within less space; therefore, it was not carefully divided into modules. Figure 1.12 shows the structure of the MS-DOS system.

Though MS-DOS has a limited structuring, there is no clear separation between the different interfaces and level of functionality. For example, application programs can directly call the basic I/O routines to read/write data on disk instead of going through a series of interfaces. This exemption makes the MS-DOS system susceptible to malicious programs that may lead to system crash. Moreover, due to the lack of hardware protection and dual-mode operation in the Intel 8088 system (for which MS-DOS system was developed), the base hardware was directly accessible to the application programs.

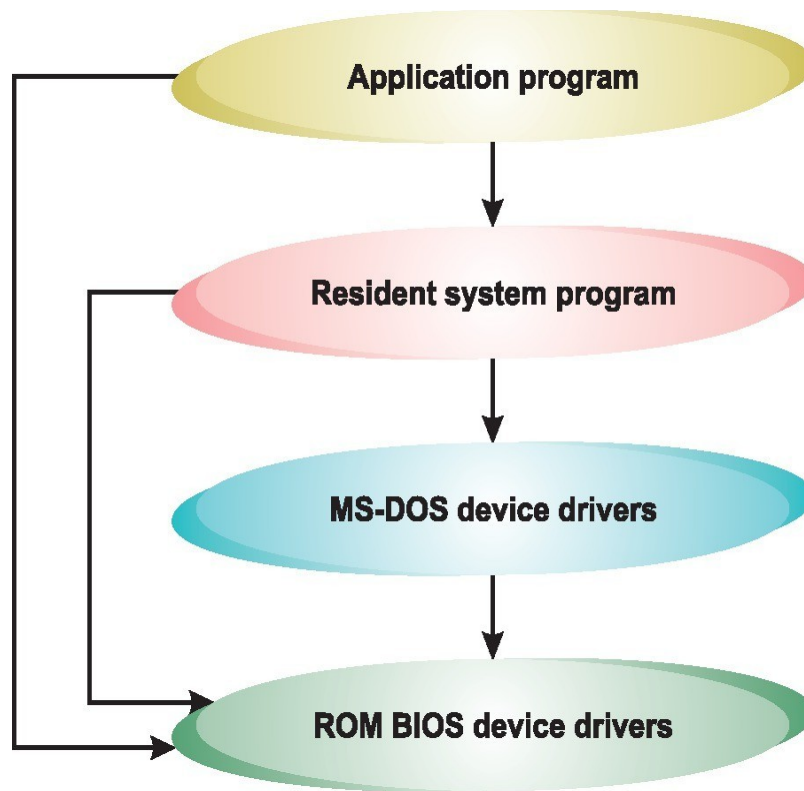


Fig. 1.12 Structure of the MS-DOS System

Advantages:

- The main advantage of having a monolithic architecture of the operating system is that it provides CPU scheduling, memory management, File management, etc through system calls.
- In a single address space, the entire large process is running.
- It is a single static binary file.

Disadvantages:

- The main disadvantage is that all components are interdependent and when one of them fails the entire system fails.
- In case the user has to add a new service or functionality the entire operating system needs to be changed.

Layered Approach

In the layered approach, the operating system is organized as a hierarchy of layers with each layer built on the top of the layer below it.

Components with similar functionalities are grouped to form a layer and in this way, total $n+1$ layers are constructed and counted from 0 to n .

Each layer has a well-defined function and comprises data structures and a set of routines.

The layers are implemented according to the following rule:

- Each layer can communicate with all of its lower layers but not with its upper layer i.e. any i^{th} layer can communicate with all layers from 0 to $i-1$ but not with the $i+1^{\text{th}}$ layer.
- Each layer is designed in such a way that it will only need the functionalities which are present in itself or the layers below it.

The topmost layer is the user interface, while the bottommost layer is the hardware.

Example: Windows XP and LINUX implements some level of layering.

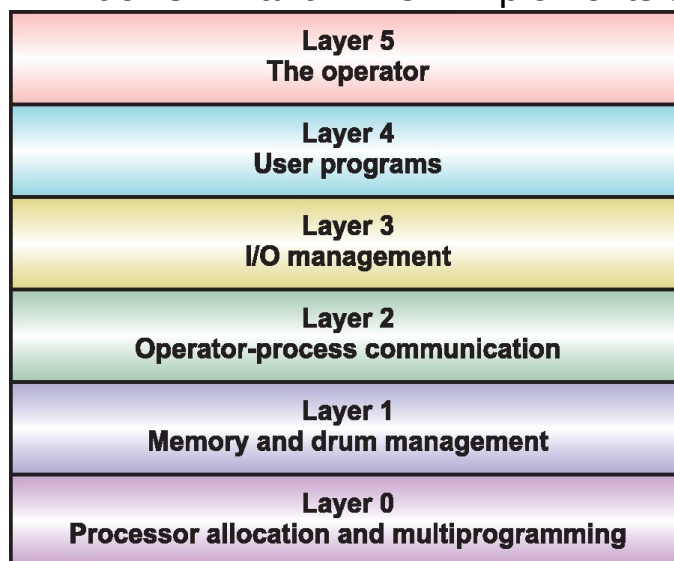
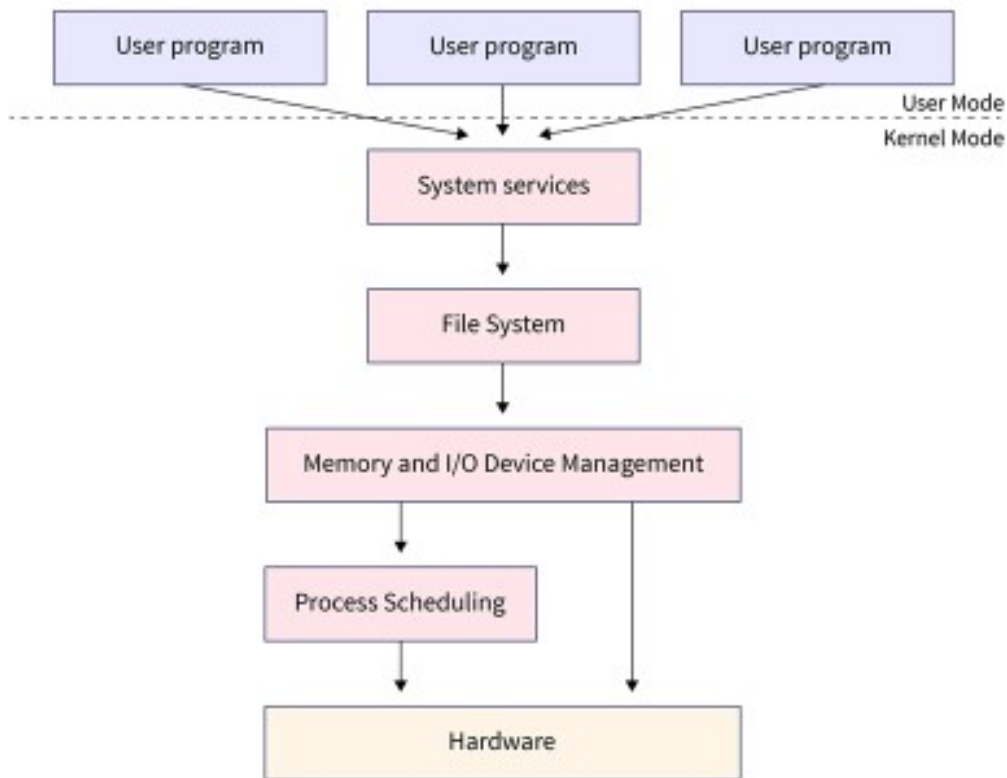


Fig. 1.13 Layers in THE System

'THE system' was the first layer-based operating system developed in

1968 by E.W. Dijkstra and his students. This operating system consisted of six layers (0-5) and each layer had a predefined function as shown in Figure 1.13.



1) Hardware: This layer is the **lowest layer** in the layered operating system architecture, this layer is **responsible for the coordination with peripheral devices** such as keyboards, mice, scanners etc.

2) CPU scheduling: This layer is **responsible for process scheduling**, multiple queues are used for scheduling. Process entering the system are kept in the **job queue** while those which are ready to be executed are put into the **ready queue**. It manages the processes which are to be kept in the CPU and those which are to be kept out of the CPU.

3) Memory Management: This layer handles the aspect of memory management i.e. **moving the processes from the secondary to primary memory for execution and vice-versa**. There are memories like RAM, and ROM. RAM is the memory where our processes run. They are moved to the RAM for execution and when they exit they are removed from RAM.

4) Process Management: This layer is **responsible for managing the various processes** i.e. assigning the CPU to those processes on a **priority**

basis for their execution. Process management uses many scheduling algorithms for prioritizing the processes for execution such as the Round-Robin algorithm, FCFS(First Come First Serve), SJF(Shortest Job First), etc.

5) I/O Buffer: Buffering is the temporary storage of data and I/O Buffer means that the data input is first buffered before storing it in the secondary memory.

All I/O devices have buffers attached to them for the temporary storage of the input data because it cannot be stored directly in the secondary storage as the speed of the I/O devices is slow as compared to the processor.

6) User Programs: This is the application layer of the layered architecture of the operating system, it deals with all the application programs running eg games, browsers, words, etc. It is the highest layer of layered architecture.

The layered design of the operating system provides some **benefits**, which are as follows:

- Layered architecture of the operating system **provides modularity because each layer is programmed to perform its own tasks only.**

Each layer has its own functionalities and it is concerned with itself only and other layers are abstracted from it.

- Since the layered architecture has independent components changing or updating one of them will not affect the other component or the entire operating system will not stop working, hence it is **easy to debug and update.**
- The user can easily access the services of the hardware layer but cannot access the hardware layer itself because it is the innermost layer.
- It simplifies the debugging and verification of the system. As the lowest layer uses merely the base hardware, it can be debugged without pertaining to the rest of the system. Once it has been verified, its correct functioning can be assumed while the second layer is being verified. Similarly, each higher level layer can be debugged independent of the lower layer. If during verification, any bug is found, it will be on the layer being debugged as lower layers have already been verified.
- It supports information hiding. Each higher level layer is required

to know only what operations the lower layers provide and not how they are being implemented.

The layered approach has some **limitations** too, which are as follows:

- Layered architecture is **complex in implementation** because one layer may use the services of the other layer. As each higher level layer is allowed to use only its lower level layers, the layers must be defined carefully.

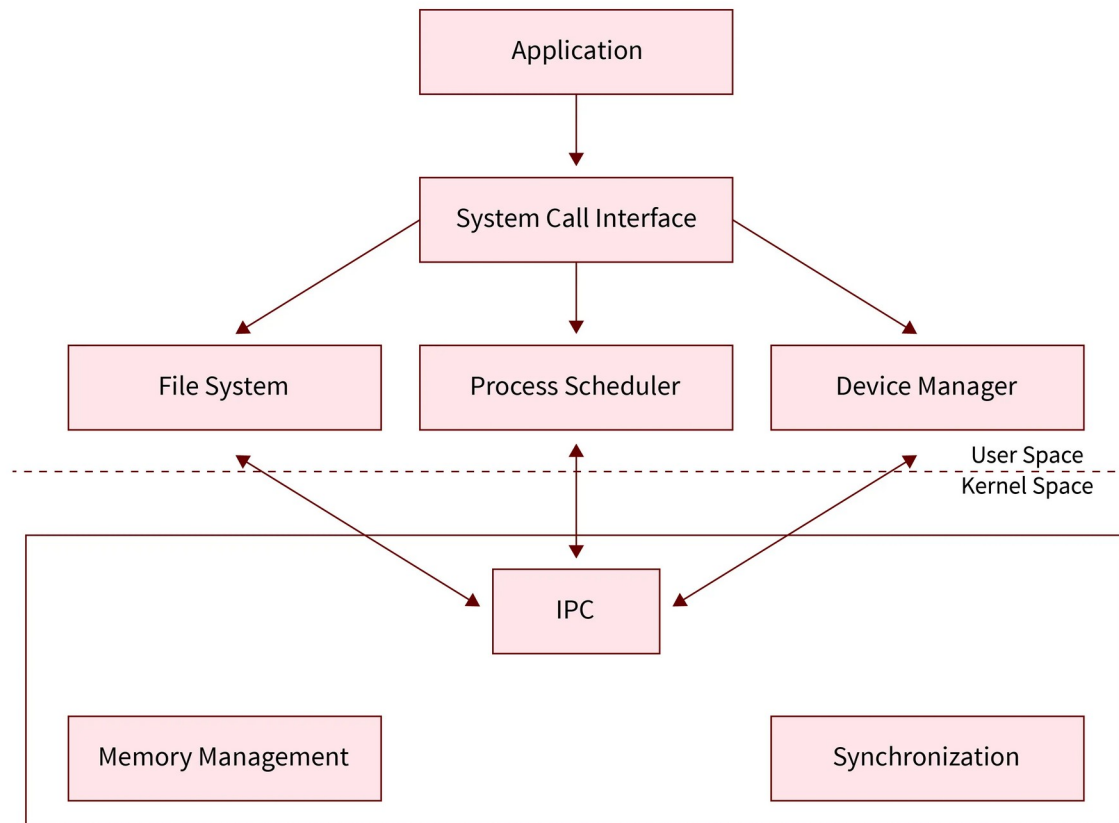
For **example**, the device driver of the physical disk must be defined at a layer below the one containing memory-management routines. This is because memory management needs to use the physical disk.

- In a layered architecture, if one layer wants to communicate with another it has to send a request(**System Call**) which has to pass through a number of layers in between before the action could be taken. This **increases response time** causing **system overhead to increase** leading to inefficiency in the system.

Microkernels

Initially, the size of kernel was small; with Berkley UNIX (BSD) began the era of large monolithic kernels. The **monolithic kernel** runs every basic system service like scheduling, inter-process communication, file management, process and memory management, device management, etc., in the kernel space itself. The inclusion of all basic services in kernel space increased the size of the kernel. In addition, these kernels were difficult to extend and maintain. The addition of new features required the recompilation of the whole kernel, which was time and resource consuming.

To overcome these problems, an approach called **microkernel** was developed that emphasized on modularizing the kernel.



The idea is to **remove the less essential components from the kernel and keeping only a subset of mechanisms typically included in a kernel** thereby **reducing its size as well as number of system calls**. The **components moved outside the kernel are implemented either as system- or user-level programs**. MACH system and OS X are the examples of operating systems designed with microkernel approach.

Advantages

- Microkernel operating systems are **modular** and hence, **disturbing one of the components will not affect the other component**. Hence the **operating system can be extended easily**.
- Microkernel **offers high security and reliability as most services run as user processes rather than kernel processes**. Thus, if any of the running services fail, the rest of the system remains unaffected.
- The architecture is **compact and isolated** and hence relatively **efficient**.
- **New features can be added without recompilation**.

Note: Though in the microkernel approach, the size of the kernel was

reduced, still there is an issue regarding which services to be included in the kernel and which services to be implemented at the user level.

Disadvantages:

- ❑ **Implementing drivers** as procedures require a **function call or context switch**.
- ❑ In microkernel architecture, **providing services is costlier** than monolithic operating systems.

Module-based OS

The **module approach** employs object-oriented programming techniques to design a modular kernel.

In this approach, the operating system is organized around a **core kernel and other loadable modules** that can be **linked dynamically with the kernel** either at boot time or at run time.

The idea is to make the **kernel providing only core services while certain services can be added dynamically**.

An example of module-based operating system is **Solaris**, which consists of **core kernel and seven loadable kernel modules**: scheduling classes, file systems, loadable system calls, executable formats, streams modules, miscellaneous, and device and bus drivers.

The modular approach **is similar to the layered approach** in the sense that **each kernel module has well-defined interfaces**. However, it is **more flexible than the layered approach as each module is free to call any other module**.